

TOWARDS EIGENVECTOR PREDICTION WITH NEURAL ALGORITHMIC REASONING

Andreea Zaharia

Department of Computer Science and Technology
University of Cambridge
az396@cam.ac.uk

ABSTRACT

The eigenvectors of a graph’s adjacency matrix are valuable graph measures, as they describe how the information flows through the graph. This project explores a novel direction for differentially computing eigenvectors using neural algorithmic reasoning. The central idea is to use neural execution, as introduced by Veličković et al. (2020), to imitate the execution of the Arnoldi iteration, a standard iterative method for eigenvector approximation. We identify a suitable step-wise supervision signal and investigate the performance of graph neural networks with different architecture characteristics. Evaluating the resulting model on synthetic data, we find that it performs well at finding a basis of the Krylov subspace on many different graph types, both in distribution and out of distribution.

1 INTRODUCTION

The eigenvectors of a graph are good descriptors of how the information flows through the graph. This makes eigenvectors valuable graph measures, proven helpful in numerous research tasks, such as graph representation learning, reinforcement learning, and computer vision. Their broad applications motivate the search for ways to compute them efficiently, especially in a differentiable manner that enables leveraging the power of deep learning. Current methods for computing eigenvectors are either too computationally expensive or they are not differentiable.

Prior work has shown that graph neural networks can effectively simulate graph algorithms, using step-wise supervision (Veličković et al., 2020). For this project, we implement and analyse a graph neural network architecture that uses the principles of neural algorithmic reasoning to imitate the Arnoldi iteration method. The Arnoldi iteration is a numerical method used for approximating eigenvectors by computing an orthogonal basis of the Krylov subspace defined by the graph’s adjacency matrix and an arbitrary vector.

We set out to devise a graph neural network model and find a suitable supervision strategy that allow the network to learn to imitate a step of the Arnoldi iteration. The experiments conducted have been designed to allow evaluating if and, subsequently, how well the model converges and how well it can generalise on different types of graphs at different scales.

2 BACKGROUND

2.1 NEURAL ALGORITHMIC REASONING

Graph neural network (GNN) architectures align well with certain algorithmic tasks, so they can be used to imitate classical algorithms in a deep learning manner. Veličković et al. (2020) introduced step-wise supervision as a means of instructing the GNN to reach the desired output by respecting the intermediate computation of the algorithm all throughout its execution. This leads to improved structural alignment with the algorithm and thus to better extrapolation power to inputs at different scales (Xu et al., 2019). Thus, neural execution of graph algorithms with step-wise supervision proves highly advantageous for out-of-distribution generalisation, as it prevents over-fitting on the computation for inputs similar in size to the training set.

Veličković et al. (2020) show that GNNs can be used to imitate individual steps of graph algorithms such as breadth-first search, Bellman-Ford, and Prim’s algorithm. Building on this work, Georgiev & Lió (2020) show how neural execution can be applied to a more complex problem, exemplifying on maximum bipartite matching solved with the Ford–Fulkerson algorithm for maximum flow.

Neural algorithmic reasoning has the advantage of enabling pre-trained GNNs to imitate the execution of classical algorithms on inputs whose format and complexity would render unusable (Veličković & Blundell, 2021). Existing work provides proofs of concept for neural algorithmic reasoning in reinforcement learning, through Executed Latent Value Iteration Networks (XLVIN) (Deac et al., 2020), and representation learning, through Representation-Modulated Reasoning (Veličković et al., 2021). Furthermore, Georgiev et al. (2021) explore the explainability of such neural execution models and introduce concept-bottleneck GNNs for explainable algorithmic reasoning.

Iterative GNNs (IterGNNs), proposed by Tang et al. (2020), address the issues posed by iterative algorithms with stopping criteria, which require a variable number of steps. The architecture in this project does not build on their methods because the iterative method we employ does not have a stopping criterion; the Arnoldi iteration runs in a pre-defined number of steps which can be specified as a parameter.

2.2 EIGENVECTORS AND THEIR APPLICATIONS IN MACHINE LEARNING

Eigenvectors are all such vectors whose directions remain unchanged when a linear transformation, such as scaling, is applied to them. For an $n \times n$ matrix A , eigenvalues λ and eigenvectors $\mathbf{v}$ are all such pairs that satisfy:

$$A\mathbf{v} - \lambda\mathbf{v} = 0 \quad (1)$$

Eigenvectors have numerous use cases across a wide range of tasks in machine learning. In graph representation learning, they have been used in spectral message passing (Stachenfeld et al., 2020) and as positional encodings for nodes in a graph neural network (Dwivedi et al., 2021). Eigenvectors have plenty of other applications, some notable ones being for image segmentation in computer vision (Weiss, 1999) and for the temporal abstraction of actions in reinforcement learning (Machado et al., 2021).

2.3 MATRICES IN HESSENBERG FORM

Notable for methods addressing the eigenvalue problem is the concept of a Hessenberg matrix. A Hessenberg matrix is a square matrix that is almost tridiagonal; in the case of an upper Hessenberg matrix, all entries below the first subdiagonal are zero. Hessenberg matrices are valuable because they benefit from many of the computational optimisations that apply to tridiagonal matrices, while being far more accessible (Bulirsch et al., 2002). Any general matrix can be reduced to a Hessenberg matrix in a finite number of steps using methods such as Householder’s transformations, the Arnoldi iteration, or Givens rotations.

2.4 PROJECTION IN THE KRYLOV SUBSPACE

For $A \in \mathbb{C}^{m \times m}$ and $\mathbf{b} \in \mathbb{C}^m$, the n^{th} order Krylov subspace is defined as the linear combination of:

$$\mathcal{K}_n = \text{span} \{ \mathbf{b}, A\mathbf{b}, A^2\mathbf{b}, \dots, A^{n-1}\mathbf{b} \} \quad (2)$$

Krylov subspaces are used in projection methods for solving eigenvalue problems, large algebraic linear systems, and matrix equations (Higham et al., 2015, Chapter II.23). Of special interest are well conditioned bases of Krylov subspaces, as they maximise the transfer of information from the original algebraic system to the projection system (Liesen & Strakos, 2013, Chapter 2).

Krylov subspaces are central to many iterative methods for approximating eigenvectors, as the Krylov sequence $\{ \mathbf{b}, A\mathbf{b}, A^2\mathbf{b}, \dots \}$ converges to the dominant eigenvector of matrix A , i.e. the eigenvector corresponding to the largest eigenvalue in absolute value. Furthermore, a sequence of n vectors forming an orthogonal basis of the n^{th} order Krylov subspace spans approximations of the eigenvectors corresponding to the n largest eigenvalues in absolute value (Wikipedia, 2022).

Well-defined Krylov subspace methods are finite processes (Liesen & Strakos, 2013, Chapter 2). However, practically sound methods also need to account for issues of numerical instability caused by inexact arithmetic.

3 METHODOLOGY

3.1 THE ARNOLDI ITERATION

The Arnoldi iteration is an iterative method used for approximating the largest n eigenvalues of a general matrix, where n is the number of iterations performed. The Arnoldi algorithm reduces a given non-hermitian matrix to its upper Hessenberg form by iteratively applying orthogonal similarity transformations, proceeding column by column from an arbitrarily chosen first column (Trefethen & Bau III, 1997, Chapter 33).

Other algorithms for approximating eigenvectors exist. Some of them are direct methods, such as the Householder transformations, and must complete to give a useful result. We preferred an iterative method to a direct method because it produces useful partial results, making it easier to reason about in the neural algorithmic reasoning framework. Among the iterative methods that apply to general matrices, the Arnoldi iteration is the standard approach (Trefethen & Bau III, 1997, Chapter 33). The Arnoldi iteration uses a modified Gram–Schmidt process for orthogonalisation and it can also be seen as a numerically stable version of the power iteration method.

The Arnoldi iteration produces a unitary matrix Q_n whose columns form an orthogonal basis of the n^{th} order Krylov subspace, $\mathcal{K}_n$. These columns, $q_1, q_2, \dots, q_n$ are called the Arnoldi vectors. The matrix Q is computed recursively column by column as follows:

$$q_{n+1} = \frac{Aq_n - \sum_{i=1}^n h_{i,n} q_i}{h_{n+1,n}} \quad (3)$$

The numerical stability is achieved by subtracting, at each step, the projections of the candidate vector v onto each of the already computed Arnoldi vectors, i.e. columns of the matrix Q .

Algorithm 1 shows the pseudocode for the Arnoldi iteration.

Algorithm 1: Arnoldi iteration

Input: the adjacency matrix A

Output: the unitary matrix Q

```

1  $\mathbf{b} \leftarrow$  an arbitrary initial vector
2  $\mathbf{q}_1 \leftarrow \mathbf{b}/\|\mathbf{b}\|$ 
3
4 for  $n \leftarrow \{1, 2, 3, \dots\}$  do
5    $\mathbf{v} \leftarrow A\mathbf{q}_n$ 
6
7   for  $j \leftarrow \{1, 2, \dots, n\}$  do
8      $h_{j,n} \leftarrow \mathbf{q}_j^* \mathbf{v}$ 
9      $\mathbf{v} \leftarrow \mathbf{v} - h_{j,n} \mathbf{q}_j$ 
10  end
11
12   $h_{n+1,n} \leftarrow \|\mathbf{v}\|$ 
13   $\mathbf{q}_{n+1} \leftarrow \mathbf{v}/h_{n+1,n}$ 
14 end
```

3.2 SYNTHETIC DATA GENERATION FOR TRAINING AND VALIDATION

We train a graph neural network architecture on synthetic data representing graphs. To ease the analysis of the model’s out-of-distribution generalisation capabilities, we generate only graphs of

the same type and size for training. These are random scale-free networks with 100 nodes, generated using the Barabási-Albert preferential attachment method. The validation data is generated with the same strategy but a different seed for randomisation. Algorithm 1 guarantees normalisation of the $\{q_1, q_2, \dots, q_n\}$ vectors, which facilitates generalisation across the different graph types.

For each such graph, we run twenty steps of the Arnoldi iteration as given in Algorithm 1 and save intermediary results in a vector, which we will henceforth call *the supervision signal*. The iterative step of the Arnoldi method, given as Equation 3, is a first-order recurrence, using computation from step n to derive the values of h_{n+1} and q_{n+1} . However, with the modification for numerical stability, each iteration needs to have access to the value of all previously computed columns of the matrix Q , to subtract v 's projection onto each one of them. We used this property to design the supervision signal for the task and we chose to save for each step n from one to twenty, the predefined number of steps, a pair formed by matrix Q in its state at step $n - 1$ and matrix Q in its state at step n .

Then, this supervision signal label is used to train the model with supervision at every step. We also implemented a mechanism for injecting the supervision signal only once every few steps and using the model's own intermediary computation between these injection points. Experimenting with different values of this update frequency, we observed that such a training strategy consistently yields lower performance for this task compared to the original step-wise approach. Thus, the model is exposed to the ground truth intermediary value of Q at every step, as suggested originally by Veličković et al. (2020).

3.3 IMPLEMENTATION AND SYSTEM SETUP

The code is implemented in Python and all dependencies are in the open-source domain. The implementation uses Jraph, a JAX library for graph neural networks (Godwin et al., 2020), and Haiku, a neural network library for JAX (Hennigan et al., 2020). We use the *Weights & Biases* (WandB) tool (Biewald, 2020) for experiment tracking, plotting, and model evaluation.

We define the GNN using Jraph's implementation of the layer described in (Battaglia et al., 2018, Algorithm 1), with capabilities for updates and aggregations as shown in Figure 1.

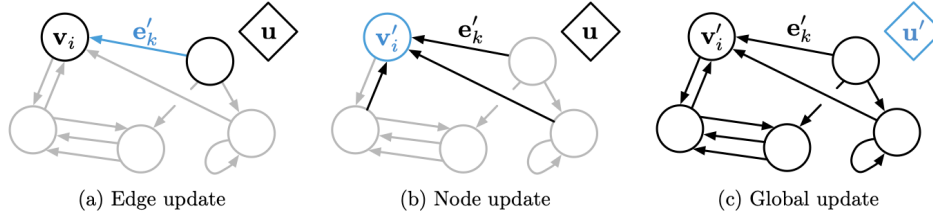


Figure 1: Updates in a Graph Network, as defined in Battaglia et al. (2018) and implemented in Jraph's GraphNetwork (Godwin et al., 2020). Image source: Battaglia et al. (2018).

The implementation is set up to benefit from GPU acceleration when a GPU is present on the system. In such a case, the NVIDIA CUDA Toolkit and the cuDNN library are also necessary dependencies, as they enable the use of Nvidia GPUs for general-purpose computation. For this project, the model was run on the CPU alone, a 10th generation 2.3 GHz 8-Core Intel Core i9.

3.4 GRAPH NEURAL NETWORK ARCHITECTURE

We use the encode-process-decode paradigm, explained in Hamrick et al. (2018), to execute the algorithm and compute the matrix of Arnoldi vectors through the GNN.

The encoder takes the input graph through an MLP and produces an embedded graph, which is then passed through the processor GNN. Experimentally, we found that deeper architectures, with two or more layers, performed significantly worse compared to the single-layer one. The processed outputs are then fed to a decoder to obtain the matrix Q of Arnoldi vectors spanning the Krylov subspace.

The processor network is a Message Passing Neural Network (MPNN) (Gilmer et al., 2017) (Equation 4), with the `sum` aggregator, modified such that it performs node updates, edge updates, and global updates. The neural networks M and U are, in our case, single-layer linear projections, with a rectified linear unit activation function (ReLU). In our experiments, we had the model compute a latent dimension of 32 features for nodes, edges, and globals separately.

$$\vec{h}_i^{(t+1)} = U \left(\vec{h}_i^{(t)}, \bigoplus_{(j,i) \in E} M \left(\vec{h}_i^{(t)}, \vec{h}_j^{(t)}, \vec{e}_{ij}^{(t)} \right) \right) \quad (4)$$

The model is trained using the Adam SGD optimiser with weight decay regularisation and an initial learning rate of 0.001, using the mean squared error as loss function. As a means of regularisation, the training of the neural network is halted using a method of early stopping with patience. The patience value dictates after how many epochs without improvement of the validation loss the training should cease.

4 EXPERIMENT

In this project we set out to investigate experimentally the following questions:

1. Is the matrix of Arnoldi vectors a suitable training signal for step-wise supervision?
2. How does the model trained on a single graph type of fixed size generalise to other graph types and sizes?

For the first question, we assess suitability based on a qualitative analysis mostly focused on the training curves. A useful supervision signal reflects in the loss curves of training and validation as a stable decrease, with the validation loss slightly larger than the training loss, until an eventual plateau in improvement.

For the second question, we generate synthetic data across a variety of different types and sizes of graphs and evaluate the model's performance based on the rolled-out mean squared error as defined in Section 4.2.

4.1 SYNTHETIC DATA FOR EVALUATION

We evaluate the model on synthetic data in the same format to that used in training and validation. For evaluation however we generate random graphs belonging to ten different popular classes of graphs:

- **Line** graphs of size N .
- **Star** graphs of size N , also known as radial structures.
- **Erdos-Rényi** random graphs (Erdos et al., 1960) of size N , with probability $1/2$ for each of the possible edges.
- Random scale-free graphs of size N , generated using the **Barabási-Albert** (Albert & Barabási, 2002) preferential attachment model.
- **Caveman** graphs of m cliques of size k , with m and k as close as possible to amount to a total graph size N .
- **Tree** graphs of size N with a power law degree distribution.
- **2D grid** graphs shaped $m \times k$, with m and k as close as possible to amount to a total graph size N .
- **Ladder** graphs of size N or $N + 1$, such that the number of nodes is even.
- **Catterpillar** graphs with a backbone of size b , drawn from a uniform distribution, and $N - b$ vertices uniformly connected to the backbone.
- **Lobster** graphs with a backbone of size b , drawn from a uniform distribution, and p , also drawn from a uniform distribution, pendent vertices uniformly connected to the backbone. The rest of $N - b - p$ vertices are connected to the p pendent vertices.

The training batches contain 32 random Barabási-Albert graphs each and the validation runs on 128 Barabási-Albert graphs. The evaluation is done on 128 graphs of each type for each size (20, 50, 100, 200, 300, and 400 nodes).

4.2 METRICS

In all stages of the computation – training, validation, and evaluation – we measure both the L1 loss and the mean squared error (MSE). In the spirit of neural execution with step-wise supervision, we are not concerned solely with the final outcome but also with how well the GNN manages to imitate the algorithm throughout its entire execution. Thus, we will evaluate the loss at every iteration, with different execution models during training and evaluation.

In training and validation, we compute the loss of the prediction with respect to the ground truth for every step of the algorithm and average the loss value across the number of iterations. As originally prompted by Veličković et al. (2020), we introduce the ground truth as training signal for every step.

In evaluation, we only give the model the ground truth for step zero and we allow it to progress with its own intermediary results for the entire computation of the algorithm. Then, rather than comparing only the final output, we compute the per-iteration loss for all steps and average them. With this approach we define a metric of how close the intermediary computation of the neural execution model is to that of the classical algorithm. In this evaluation procedure, we provide the ground truth for the very first step in order to eliminate the differences that would arise from the choice of the arbitrary first column. While different choices of these random values are harmless, they lead to different outputs and they would hence misleadingly hurt the test loss.

4.3 OUT-OF-DISTRIBUTION GENERALISATION

For each graph type, we execute both in-distribution and out-of-distribution experiments. The in-distribution scenario is achieved by testing on hundred node graphs, the same size as the graphs used in the synthetic training dataset. For out-of-distribution evaluation, we run experiments on inputs with more nodes (200 – 500), as well as with fewer nodes (20 and 50), for each graph type.

5 RESULTS AND DISCUSSION

We ran a series of experiments based on the design presented in Section 4, iterating over several modifications of the configuration. While seeking answers to the two questions posed in Section 4, we also made a series of other observations which we believed might prove useful to further efforts on neural algorithmic reasoning for eigenvector prediction or related learning tasks.

An entire experiment run, training plus evaluation, takes around three hours for the network with node, edge, and global dimensions of 32 in the latent space. We also experimented with a narrower network, with a latent dimension of 16, which proved useful for faster iteration but consistently yielded lower performance.

All plots included in this section are shown on semi-logarithmic scale because the loss value declines precipitously over a relatively low number of steps.

Figure 2 illustrates how the model successfully manages to use the given supervision signal to optimise the loss. This is a promising result for the competence of our training strategy when it comes to creating the desired inductive bias within the GNN’s update rule.

We next evaluate the model on synthetically generated graphs of ten different structural types and six different sizes, created as described in Section 4.1. The results, shown in Figure 3, make it immediately obvious that the GNN fails to provide any useful prediction on the task for star graphs, Erdős-Rényi graphs, and Barabási-Albert graphs. In stark contrast, the GNN performs quite well on the other seven graph types we evaluate it on. These three graph types that prove more challenging all have in common the fact that they are scale-free networks, i.e. their node degrees exhibit a power-law distribution, with the star graph being the most extreme case.

Both due to the importance of scale-free networks in the real-world and due to their interesting spectral properties, the spectra and eigenvectors of the adjacency matrices of scale-free networks have

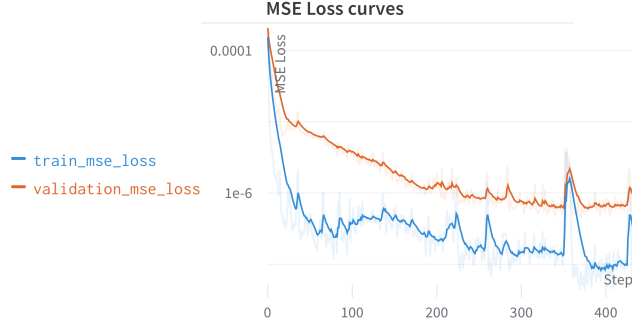


Figure 2: Training curves, measuring mean squared error (MSE) for each step of the algorithm.

been studied extensively (Goh et al., 2001; Farkas et al., 2002; Gago Álvarez, 2007; Khomyakova & Konstantinova, 2021). As their spectral density exhibits power-law long tails and an exponential decay around the center, their structural properties might explain what makes scale-free networks a particularly challenging input to out-of-distribution generalisation on this task.



Figure 3: Step-wise mean squared error computed by averaging the loss of the GNN ran with its own intermediary computation, over twenty consecutive iterations.

We further experimented with activation patterns as they are known to notably impact the behaviour of graph neural networks and contribute to the characteristics used by the GNN to progress through the task (Veyrin-Forrer et al., 2021). Experimentally, the model reached the best test performance upon removing all applications of a non-linear activation function from the GNN architecture.

In Figure 4, we find similar results to those obtained with the model with activation (Figure 3) for the seven graph types that are not scale-free networks. However, for the others, we observe vastly improved results; we would like to point out to the reader the difference in scale on the Y-axis between Figures 3 and 4. Although the performance on the star graph is still notably worse than on the other graph types, now its loss values are within the same order of magnitude as the others. On Erdős–Rényi graphs and Barabási–Albert graph, the performance is on par with that on graphs that are not scale-free, except for inputs of 300 and 400 nodes for which it is slightly worse.

Another important observation in both scenarios is that the models do not generalise well to graphs that have fewer nodes than those seen in training, irrespective of the graph type. However, the models exhibit good out-of-distribution generalisation for graphs that are two to four times larger than the ones seen in training.

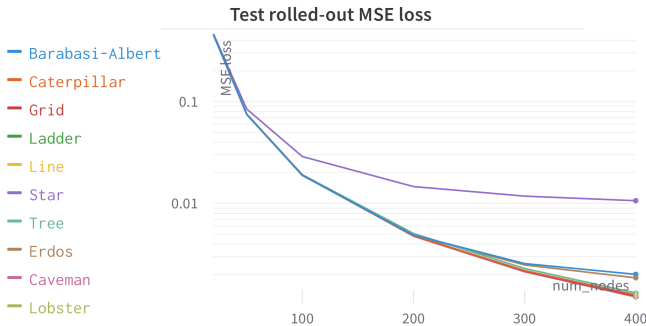


Figure 4: Step-wise mean squared error computed by averaging the loss of the GNN ran with its own intermediary computation, over twenty consecutive iterations. The model employed does not use any activation function.

6 DIRECTIONS FOR FUTURE WORK

We believe this project opens up vast opportunities for future work and this approach lends itself to numerous potential applications. Unfortunately, time constrained what we could develop and explore in the scope of the project.

An important next step for this project could be plugging this architecture into a downstream task. We attempted to explore this direction by using this vector basis computed with neural execution as positional embeddings in a GNN for graph property prediction on the ogbg-molpcba dataset (Open Graph Benchmark). The idea had been to connect the implementation presented in this report with the network for OGBG-MolPCBA given in Google (2022), using our pre-trained GNN to inject positional embeddings with the approach introduced by Dwivedi et al. (2021). Unfortunately, having faced implementation difficulties, this effort could not be completed in time for this report. Nonetheless, it remains an interesting next step to explore in this project.

Another possible direction would be to experiment with other iterative methods commonly used for eigenvalue problems and compare the experimental results with the performance obtained with the Arnoldi iteration. One such example is the inverse iteration Peters & Wilkinson (1971), method described by the iteration step:

$$x_{r+1} = \frac{(A - \mu I)^{-1} x_r}{\|(A - \mu I)^{-1} x_r\|} \quad (r = 0, 1, \dots) \quad (5)$$

One experimental observation deserving of further investigation is that the rolled-out test loss reveals that the intra-step error accumulates and amplifies over the twenty steps.

Lastly, to expand on the experiments and evaluate the breadth of potential applications this approach might prove suitable for, one could synthetically introduce perturbations on the adjacency matrix to simulate the scenario when downstream tasks do not come with correct adjacency matrices.

7 CONCLUSION

This project investigated eigenvector approximation with the Arnoldi iteration through neural algorithmic reasoning. We trained and evaluated a graph neural network architecture on synthetic graph data, with step-wise supervision. We identified several architecture characteristics which significantly impact performance at this task, such as the number of layers and the activation pattern. A significant finding of this exploration is that, using the intermediary matrix of Arnoldi vectors as supervision signal, the model successfully learns to predict a basis of the Krylov subspace of the user-specified order. This is a promising result as a potential application in the form of learnable positional embeddings in a downstream task, as well as for the broader research area pertaining to the neural execution of iterative methods.

ACKNOWLEDGEMENTS

This project originated from Andreea Deac, to whom I am very grateful for all the kind support and guidance that made this work possible. I would also like to thank Dr Petar Veličković and Professor Pietro Lió for guiding my first steps in the world of graph representation learning.

REFERENCES

- Réka Albert and Albert-László Barabási. Statistical mechanics of complex networks. *Reviews of modern physics*, 74(1):47, 2002.
- Peter W Battaglia, Jessica B Hamrick, Victor Bapst, Alvaro Sanchez-Gonzalez, Vinicius Zambaldi, Mateusz Malinowski, Andrea Tacchetti, David Raposo, Adam Santoro, Ryan Faulkner, et al. Relational inductive biases, deep learning, and graph networks. *arXiv preprint arXiv:1806.01261*, 2018.
- Lukas Biewald. Experiment tracking with weights and biases, 2020. URL <https://www.wandb.com/>. Software available from wandb.com.
- Roland Bulirsch, Josef Stoer, and J Stoer. *Introduction to numerical analysis*, volume 3. Springer, 2002.
- Andreea Deac, Petar Veličković, Ognjen Milinković, Pierre-Luc Bacon, Jian Tang, and Mladen Nikolić. XLVIN: eXecuted Latent Value Iteration Nets. *arXiv:2010.13146 [cs, stat]*, December 2020. URL <http://arxiv.org/abs/2010.13146>. arXiv: 2010.13146.
- Vijay Prakash Dwivedi, Anh Tuan Luu, Thomas Laurent, Yoshua Bengio, and Xavier Bresson. Graph neural networks with learnable structural and positional representations. *arXiv preprint arXiv:2110.07875*, 2021.
- Paul Erdos, Alfréd Rényi, et al. On the evolution of random graphs. *Publ. Math. Inst. Hung. Acad. Sci*, 5(1):17–60, 1960.
- I Farkas, I Derényi, Hawoong Jeong, Z Neda, ZN Oltvai, E Ravasz, A Schubert, A-L Barabási, and T Vicsek. Networks in life: Scaling properties and eigenvalue spectra. *Physica A: Statistical Mechanics and its Applications*, 314(1-4):25–34, 2002.
- Silvia Gago Álvarez. Eigenvalue distribution in scale free graphs. 2007.
- Dobrik Georgiev and Pietro Lió. Neural bipartite matching. *arXiv preprint arXiv:2005.11304*, 2020.
- Dobrik Georgiev, Pietro Barbiero, Dmitry Kazhdan, Petar Veličković, and Pietro Lió. Algorithmic Concept-based Explainable Reasoning. *arXiv:2107.07493 [cs]*, July 2021. URL <http://arxiv.org/abs/2107.07493>. arXiv: 2107.07493.
- Justin Gilmer, Samuel S Schoenholz, Patrick F Riley, Oriol Vinyals, and George E Dahl. Neural message passing for quantum chemistry. In *International conference on machine learning*, pp. 1263–1272. PMLR, 2017.
- Jonathan Godwin, Thomas Keck, Peter Battaglia, Victor Bapst, Thomas Kipf, Yujia Li, Kimberly Stachenfeld, Petar Veličković, and Alvaro Sanchez-Gonzalez. Jraph: A library for graph neural networks in jax., 2020. URL <http://github.com/deepmind/jraph>.
- K-I Goh, Byungnam Kahng, and Doochul Kim. Spectra and eigenvectors of scale-free networks. *Physical Review E*, 64(5):051903, 2001.
- Google. Predicting biological activities of molecules with graph neural networks · google/flax, 2022. URL <https://github.com/google/flax>.
- Jessica B Hamrick, Kelsey R Allen, Victor Bapst, Tina Zhu, Kevin R McKee, Joshua B Tenenbaum, and Peter W Battaglia. Relational inductive bias for physical construction in humans and machines. *arXiv preprint arXiv:1806.01203*, 2018.

- Tom Hennigan, Trevor Cai, Tamara Norman, and Igor Babuschkin. Haiku: Sonnet for JAX, 2020. URL <http://github.com/deepmind/dm-haiku>.
- Nicholas J Higham, Mark R Dennis, Paul Glendinning, Paul A Martin, Fadil Santosa, and Jared Tanner. *The Princeton companion to applied mathematics*. Princeton University Press Princeton, NJ, USA:, 2015.
- Ekaterina Khomyakova and Elena V. Konstantinova. Catalogue of the Star graph eigenvalue multiplicities. *Arabian Journal of Mathematics*, 10(1):115–119, April 2021. ISSN 2193-5343, 2193-5351. doi: 10.1007/s40065-019-00271-z. URL <https://link.springer.com/10.1007/s40065-019-00271-z>.
- Jörg Liesen and Zdenek Strakos. *Krylov subspace methods: principles and analysis*. Oxford University Press, 2013.
- Marlos C Machado, Andre Barreto, and Doina Precup. Temporal abstraction in reinforcement learning with the successor representation. *arXiv preprint arXiv:2110.05740*, 2021.
- Open Graph Benchmark. Graph Property Prediction | Open Graph Benchmark. URL <https://ogb.stanford.edu/docs/graphprop/>.
- Gwendoline Peters and James H Wilkinson. The calculation of specified eigenvectors by inverse iteration. In *Handbook for Automatic Computation*, pp. 418–439. Springer, 1971.
- Kimberly Stachenfeld, Jonathan Godwin, and Peter Battaglia. Graph networks with spectral message passing. *arXiv preprint arXiv:2101.00079*, 2020.
- Hao Tang, Zhiao Huang, Jiayuan Gu, Bao-Liang Lu, and Hao Su. Towards scale-invariant graph-related problem solving by iterative homogeneous gnns. *Advances in Neural Information Processing Systems*, 33:15811–15822, 2020.
- Lloyd N Trefethen and David Bau III. *Numerical linear algebra*, volume 50. Siam, 1997.
- Petar Veličković and Charles Blundell. Neural algorithmic reasoning. *Patterns*, 2(7):100273, 2021.
- Petar Veličković, Matko Bošnjak, Thomas Kipf, Alexander Lerchner, Raia Hadsell, Razvan Pascanu, and Charles Blundell. Reasoning-modulated representations. *arXiv preprint arXiv:2107.08881*, 2021.
- Petar Veličković, Rex Ying, Matilde Padovano, Raia Hadsell, and Charles Blundell. Neural Execution of Graph Algorithms. *arXiv:1910.10593 [cs, stat]*, January 2020. URL <http://arxiv.org/abs/1910.10593>. arXiv: 1910.10593.
- Luca Veyrin-Forrer, Ataollah Kamal, Stefan Duffner, Marc Plantevit, and Céline Robardet. On GNN explainability with activation patterns. 2021.
- Yair Weiss. Segmentation using eigenvectors: a unifying view. In *Proceedings of the seventh IEEE international conference on computer vision*, volume 2, pp. 975–982. IEEE, 1999.
- Wikipedia. Arnoldi iteration, February 2022. URL https://en.wikipedia.org/w/index.php?title=Arnoldi_iteration&oldid=1071281738. Page Version ID: 1071281738.
- Keyulu Xu, Jingling Li, Mozhi Zhang, Simon S Du, Ken-ichi Kawarabayashi, and Stefanie Jegelka. What can neural networks reason about? *arXiv preprint arXiv:1905.13211*, 2019.